

GATE Data Science & Artificial Intelligence (DA)

Topic-Wise Question Bank: Algorithms & Uninformed Search

10 Selected Previous Year Questions (GATE DA / CS) + 10 Targeted GATE-Level Practice Questions

Syllabus Coverage:

1. **Topic 1:** Logarithmic Loops Complexity Analysis
2. **Topic 2:** Binary Search Variations & Multi-Array Complexity Tracing
3. **Topic 3:** Uninformed Search: Depth-Limited Search (DLS) & Iterative Deepening Search (IDS)
4. **Topic 4:** Differentiating $\mathcal{O}(\log n)$ vs $\mathcal{O}(\log^2 n)$ Asymptotic Traps

Part I: Previous Year Questions (PYQs)

Q1. Logarithmic Loop Variable Scaling [GATE DA 2024 • NAT • 1 Mark] Consider the following Python code snippet:

```
def compute(n):
    count = 0
    i = 1
    while i < n:
        count += 1
        i = i * 3
    return count
```

For $n = 243$, what is the exact return value of `compute(243)`?

Answer: 5

Solution / Explanation:

The loop initializes $i = 1$ and multiplies i by 3 in each iteration until $i \geq n$.

- Iteration 1: $i = 1 \times 3 = 3$, $\text{count} = 1$
- Iteration 2: $i = 3 \times 3 = 9$, $\text{count} = 2$
- Iteration 3: $i = 9 \times 3 = 27$, $\text{count} = 3$
- Iteration 4: $i = 27 \times 3 = 81$, $\text{count} = 4$
- Iteration 5: $i = 81 \times 3 = 243$, $\text{count} = 5$

Next, $i = 243$, which violates $i < 243$. The loop terminates. Thus, the function returns $\lceil \log_3(243) \rceil = 5$.

Q2. Recurrence of Standard Binary Search

[GATE DA

Sample 2024 • MCQ • 1 Mark] Which of the following correctly describes the recurrence relation for standard binary search on an array of n sorted numbers, where c is a constant?

- (A) $T(n) = 2T(n/2) + c$
- (B) $T(n) = T(n/2) + c$

- (C) $T(n) = T(n - 1) + c$
 (D) $T(n) = 2T(n - 1) + c$

Answer: (B)

Solution / Explanation:

In binary search, the search key is compared with the middle element. Depending on the result, the problem size is halved ($n/2$), discarding the remaining half entirely. The comparison and index calculation take constant time c . Hence, $T(n) = T(n/2) + c$, which solves to $T(n) = \Theta(\log n)$ by the Master Theorem.

Q3. Iterative Deepening Node Expansion

[GATE DA Sample 2024 • MCQ • 2 Marks]

Let b be the uniform branching factor of a search tree. If the optimal goal node is located at depth d from the initial state, how many times will the initial (root) state be expanded in the worst case by Iterative Deepening Depth-First Search (IDDFS)?

- (A) 1
 (B) d
 (C) $d + 1$
 (D) b^d

Answer: (C)

Solution / Explanation:

IDDFS performs depth-limited searches starting from limit $L = 0$ up to limit $L = d$ when the goal is discovered.

- At limit $L = 0$: Root is generated and evaluated/expanded once.
- At limit $L = 1$: Root is expanded again down to depth 1.
- At limit $L = k$: Root is expanded again.
- Continuing through limit $L = d$: Root is expanded again.

The total number of times the initial root node is expanded is $(d - 0 + 1) = d + 1$.

Q4. Nested Loop: Inner Step Traps

[GATE CS 2021 • NAT • 2 Marks]

Consider the following pseudo-code segment where n is a power of 2:

```
count = 0;
for (int i = 1; i <= n; i = i * 2) {
    for (int j = 1; j <= i; j = j * 2) {
        count++;
    }
}
```

If $n = 64$, what is the final value of count?

Answer: 28

Solution / Explanation:

Let $n = 2^k$. For $n = 64$, $k = 6$. The outer loop variable i assumes values $2^0, 2^1, 2^2, \dots, 2^6$. For a given $i = 2^p$ (where $0 \leq p \leq 6$), the inner loop variable j takes values $2^0, 2^1, \dots, 2^p$. The number of times the inner loop runs for a fixed p is $p + 1$. Summing across all outer loop values:

$$\text{count} = \sum_{p=0}^6 (p+1) = 1 + 2 + 3 + 4 + 5 + 6 + 7 = \frac{7 \times 8}{2} = 28.$$

In asymptotic terms, this loop family runs in $\Theta(\log^2 n)$ time.

Q5. Binary Search Midpoint Step Count

[GATE CS 2017 • NAT • 1

Mark] A sorted array contains 1024 distinct integers. In the worst case, how many comparisons with array elements are required to determine whether a target key is present using the standard binary search algorithm (where one three-way comparison counts as 1)?

Answer: 11

Solution / Explanation:

In an array of length $n = 1024$, the maximum number of comparisons performed along any branch of the binary search decision tree is $\lfloor \log_2 n \rfloor + 1$.

$$\lfloor \log_2(1024) \rfloor + 1 = 10 + 1 = 11.$$

Q6. Properties of Uninformed Search Strategies

[GATE CS 2018 / DA 2024 Scope • MSQ •

2 Marks] Which of the following statements is/are **TRUE** regarding uninformed search strategies on an infinite tree with finite branching factor $b \geq 2$?

- (A) Depth-First Search (DFS) is complete.
- (B) Breadth-First Search (BFS) is complete and optimal when step costs are equal.
- (C) Depth-Limited Search (DLS) with limit $l < d$ (where d is the shallowest goal depth) will never find the goal.
- (D) Iterative Deepening Search (IDS) has asymptotic space complexity $\mathcal{O}(bd)$ where d is the depth of the shallowest solution.

Answer: (B, C, D)

Solution / Explanation:

- (A) is False: DFS can get trapped down an infinite path and fail to terminate.
- (B) is True: BFS systematically explores layer by layer, guaranteeing completeness and shallowest-depth optimality.
- (C) is True: If the cut-off limit l is strictly less than the goal depth d , DLS terminates with failure without ever reaching depth d .
- (D) is True: IDS uses depth-first search under the hood, retaining only the path from root to current node and siblings along the path, requiring $\mathcal{O}(bd)$ space.

Q7. Independent Dual Logarithmic Loops [GATE CS 2019 • MCQ • 1 Mark] Analyze the time complexity of the following algorithm snippet in terms of input parameter n :

```
k = 1
while k <= n:
    k = k * 2
m = 1
while m <= n:
    m = m * 3
```

- (A) $\Theta(\log^2 n)$
- (B) $\Theta(n)$
- (C) $\Theta(\log n)$
- (D) $\Theta(\sqrt{n})$

Answer: (C)

Solution / Explanation:

The two while loops are sequential (not nested).

- First loop: k doubles each step $\implies \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$ iterations.
- Second loop: m triples each step $\implies \lfloor \log_3 n \rfloor + 1 = \Theta(\log n)$ iterations.

Total time = $\Theta(\log n) + \Theta(\log n) = \Theta(\log n)$.

Q8. Identifying $\mathcal{O}(\log n)$ vs $\mathcal{O}(\log^2 n)$ Code Structure [GATE CS 2022 • MCQ • 2 Marks] Consider the following two functions F_1 and F_2 :

```
def F1(n):
    c = 0; i = n
    while i > 1:
        j = 1
        while j < n:
            c += 1; j = j * 2
        i = i // 2
    return c
```

```
def F2(n):
    c = 0; i = n
    while i > 1:
        j = 1
        while j < i:
            c += 1; j = j * 2
        i = i // 2
    return c
```

Which of the following characterizations is correct regarding their running times?

- (A) Both F_1 and F_2 run in $\Theta(\log^2 n)$
- (B) F_1 runs in $\Theta(\log^2 n)$, but F_2 runs in $\Theta(\log n)$
- (C) F_1 runs in $\Theta(n)$, while F_2 runs in $\Theta(\log^2 n)$
- (D) F_1 runs in $\Theta(\log n)$, while F_2 runs in $\Theta(\log^2 n)$

Answer: (A)

Solution / Explanation:

Analysis of F_1 : The outer loop runs $\lfloor \log_2 n \rfloor$ times. In every iteration, the inner loop bounds $j < n$, taking exactly $\lceil \log_2 n \rceil$ iterations. Total time: $\Theta(\log n \times \log n) = \Theta(\log^2 n)$.

Analysis of F_2 : In step k (where $i = n/2^k$), the inner loop runs $\log_2(n/2^k) = \log_2 n - k$ times. The total operations are:

$$\sum_{k=0}^{\log_2 n - 1} (\log_2 n - k) = \sum_{m=1}^{\log_2 n} m = \frac{\log_2 n (\log_2 n + 1)}{2} = \Theta(\log^2 n).$$

Both functions run in $\Theta(\log^2 n)$.

Q9. Median Search via Two Sorted Arrays

[GATE CS 2016 • MCQ • 2 Marks] Let A and

B be two sorted arrays, each containing n distinct elements. The optimal time complexity to find the median of the merged set of $2n$ elements using a modified binary search procedure is:

- (A) $\Theta(1)$
- (B) $\Theta(\log n)$
- (C) $\Theta(n)$
- (D) $\Theta(n \log n)$

Answer: (B)

Solution / Explanation:

By comparing the medians of A and B , one can discard half of array A and half of array B in each recursive step. The recurrence is $T(n) = T(n/2) + \mathcal{O}(1)$, which solves to $\Theta(\log n)$ using divide-and-conquer binary partitioning.

Q10. Space Complexity of DLS vs IDS

[GATE DA 2025 Model / AI Curricula • MCQ • 1 Mark]

In a tree search problem with branching factor b and maximum depth m , what is the asymptotic memory (space) required by Depth-Limited Search with depth limit L ?

- (A) $\mathcal{O}(b^L)$
- (B) $\mathcal{O}(b \cdot L)$
- (C) $\mathcal{O}(L^b)$
- (D) $\mathcal{O}(m)$

Answer: (B)

Solution / Explanation:

Depth-Limited Search uses DFS traversal down to a cutoff depth L . Because it only keeps track of nodes along the current path from the root to depth L and unexpanded siblings at each level, the space required is linear in the cutoff depth and branching factor: $\mathcal{O}(b \cdot L)$.

Part II: 10 GATE-Level Practice Questions

Practice Q1. Variable Step Exponentiation Loop

[GATE DA Standard • NAT • 2 Marks]

Consider the following Python algorithm where $n = 65536$:

```
def compute_ops(n):  
    ans = 0  
    p = 2  
    while p <= n:  
        ans += 1  
        p = p * p  
    return ans
```

What is the return value of `compute_ops(65536)`?

Answer: 5

Solution / Explanation:

Let us trace the values of p after each iteration:

- Initialization: $p = 2$
- Iteration 1: $p \leq 65536$ holds. $p \leftarrow 2^2 = 4$, $\text{ans} = 1$
- Iteration 2: $p \leq 65536$ holds. $p \leftarrow 4^2 = 16 = 2^4$, $\text{ans} = 2$
- Iteration 3: $p \leq 65536$ holds. $p \leftarrow 16^2 = 256 = 2^8$, $\text{ans} = 3$
- Iteration 4: $p \leq 65536$ holds. $p \leftarrow 256^2 = 65536 = 2^{16}$, $\text{ans} = 4$
- Iteration 5: $p = 65536 \leq 65536$ holds. $p \leftarrow (65536)^2 = 2^{32}$, $\text{ans} = 5$
- Iteration 6: $p = 2^{32} > 65536$. Loop terminates.

In general, after k squaring steps, $p = 2^{2^k}$. Thus, $2^{2^k} > n \implies 2^k > \log_2 n \implies k = \lceil \log_2(\log_2 n) \rceil + 1$. For $n = 65536 = 2^{16}$, the while loop executes for $p \in \{2, 4, 16, 256, 65536\}$, yielding exactly 5 iterations.

Practice Q2. Binary Search in 2D Row-Column Sorted Matrix

[GATE DA Standard

• MCQ • 2 Marks] Consider an $n \times n$ matrix where each row is sorted in strictly ascending order from left to right, and each column is sorted in strictly ascending order from top to bottom. What is the tightest worst-case time complexity to search for a target element?

- (A) $\Theta(\log n)$
- (B) $\Theta(n)$
- (C) $\Theta(n \log n)$
- (D) $\Theta(\log^2 n)$

Answer: (B)

Solution / Explanation:

Starting from the top-right corner ($row = 0, col = n - 1$):

- If $matrix[row][col] == target$, key is found.
- If $matrix[row][col] > target$, target cannot be in current column $\implies col \leftarrow col - 1$.
- If $matrix[row][col] < target$, target cannot be in current row $\implies row \leftarrow row + 1$.

Each step decrements col or increments row . At most $2n$ steps are made. Hence, the worst-case time complexity is $\Theta(n)$. Note: Applying 1D binary search on all rows independently yields $\mathcal{O}(n \log n)$, which is suboptimal.

Practice Q3. Binary Search Floor Index Variant [GATE DA Standard • MCQ • 1 Mark] Given a sorted array $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$ and the following binary search variant designed to return the index of the greatest element smaller than or equal to $K = 20$:

```
low = 0; high = len(A) - 1; ans = -1
while low <= high:
    mid = (low + high) // 2
    if A[mid] <= K:
        ans = mid
        low = mid + 1
    else:
        high = mid - 1
```

What is the final value of ans ?

- (A) 3
- (B) 4
- (C) 5
- (D) 16

Answer: (B)

Solution / Explanation:

Let us trace:

- Step 1: $low = 0, high = 9 \implies mid = 4$. $A[4] = 16 \leq 20$. Update $ans = 4, low = 5$.
- Step 2: $low = 5, high = 9 \implies mid = 7$. $A[7] = 56 > 20$. Update $high = 6$.
- Step 3: $low = 5, high = 6 \implies mid = 5$. $A[5] = 23 > 20$. Update $high = 4$.
- Loop terminates since $low(5) > high(4)$.

The value stored in ans is 4 (corresponding to element $A[4] = 16$).

Practice Q4. Exponential Search Probe Count [GATE DA Standard • NAT • 2 Marks]

An unbounded sorted array contains a key x at index $k = 100$. The Exponential Search algorithm finds the range $[2^{i-1}, 2^i]$ containing k by repeated doubling $(1, 2, 4, 8, \dots)$, and then applies standard binary search over that bounded range. What is the total number of comparisons made to locate x ?

Answer: 14

Solution / Explanation:

Phase 1 (Range Finding): The algorithm tests indices $2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 8, 2^4 = 16, 2^5 = 32, 2^6 = 64, 2^7 = 128$. At index 64 (< 100) and index 128 (> 100), the bound is established between [64, 128]. Total comparisons in Phase 1: 8 comparisons (probes 1, 2, 4, 8, 16, 32, 64, 128).

Phase 2 (Binary Search): The search interval has length $128 - 64 + 1 = 65$ elements. Standard binary search over 65 elements takes $\lceil \log_2(65) \rceil + 1 = 6 + 1 = 7$ comparisons in the worst case, or 6 comparisons when matching on exact boundary tests. Specifically, $\lceil \log_2(65) \rceil = 7$. Probing: $mid = 96 (< 100)$, $mid = 112 (> 100)$, $mid = 104 (> 100)$, $mid = 100$ (Found in 6 comparisons). Total comparisons = $8 + 6 = 14$.

Practice Q5. Total Generated Nodes in IDS [GATE DA Standard • NAT • 2 Marks] A uniform search tree has a branching factor of $b = 3$. The goal node is positioned at depth $d = 3$. Iterative Deepening Search (IDS) is executed starting from limit $L = 0$ up to limit $L = 3$. What is the total number of node generations across all iterations? (Assume root is at depth 0).

Answer: 53

Solution / Explanation:

In IDS, nodes at depth i are generated $(d - i + 1)$ times:

$$N_{\text{total}} = \sum_{i=0}^d (d - i + 1) b^i$$

For $b = 3$ and $d = 3$:

- Depth 0: $3^0 \times (3 - 0 + 1) = 1 \times 4 = 4$
- Depth 1: $3^1 \times (3 - 1 + 1) = 3 \times 3 = 9$
- Depth 2: $3^2 \times (3 - 2 + 1) = 9 \times 2 = 18$
- Depth 3: $3^3 \times (3 - 3 + 1) = 27 \times 1 = 27$ (up to goal level)

Summing all generated nodes: $4 + 9 + 18 + 27 = 53$.

Practice Q6. Theoretical Invariants of DLS and IDS [GATE DA Standard • MSQ • 2 Marks] Which of the following statements is/are **CORRECT** regarding Depth-Limited Search (DLS) and Iterative Deepening Search (IDS)?

- (A) IDS avoids the overhead of queue storage by trading modest redundant computation at shallow levels.
- (B) The asymptotic time complexity of IDS on a tree with branching factor $b > 1$ and goal at depth d is $\mathcal{O}(b^d)$.
- (C) DLS is guaranteed to be optimal if the cutoff limit is equal to the solution depth d .
- (D) In terms of memory consumption, IDS outperforms standard Breadth-First Search (BFS) by requiring only $\mathcal{O}(bd)$ space instead of $\mathcal{O}(b^d)$.

Answer: (A, B, D)

Solution / Explanation:

- (A) is Correct: IDS re-evaluates upper nodes to keep memory linear $\mathcal{O}(bd)$.
- (B) is Correct: The ratio of nodes generated by IDS vs BFS is $\frac{b}{b-1}$. For $b \geq 2$, it is bounded by a constant factor, yielding $\mathcal{O}(b^d)$.
- (C) is Incorrect: Even if limit $l = d$, DLS behaves as DFS and may explore sub-optimal paths to depth d on the left side of the tree before reaching an optimal goal elsewhere at depth $\leq d$.
- (D) is Correct: BFS stores the frontier requiring $\mathcal{O}(b^d)$ space, whereas IDS stores only the active DFS recursion branch requiring $\mathcal{O}(bd)$ space.

Practice Q7. Inner Dependent Loop Asymptotic Trap [GATE DA Standard • MCQ • 2 Marks]

Consider the following nested loop construct:

```
ans = 0
for i in range(1, n + 1):
    j = 1
    while j <= i:
        ans += 1
        j = j * 2
```

The tight asymptotic time complexity of this code segment is:

- (A) $\Theta(n)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(\log^2 n)$
- (D) $\Theta(n^2)$

Answer: (B)

Solution / Explanation:

For a fixed index i , the while loop runs $\lfloor \log_2 i \rfloor + 1$ times. Summing across all i from 1 to n :

$$T(n) = \sum_{i=1}^n (\lfloor \log_2 i \rfloor + 1) = \sum_{i=1}^n \Theta(\log i) = \Theta(\log(n!)) = \Theta(n \log n)$$

by Stirling's approximation. This is a common trap where students mistake the outer loop running n times and inner running $\log i$ for $\Theta(\log^2 n)$.

Practice Q8. Asymptotic Distinction: $\log n$ vs $\log^2 n$ [GATE DA Standard • MSQ • 2 Marks] Let

$f(n) = \log_2(n^8)$ and $g(n) = (\log_2 n)^2$. Which of the following statements is/are TRUE?

- (A) $f(n) = \mathcal{O}(g(n))$
- (B) $g(n) = \mathcal{O}(f(n))$
- (C) $f(n) = \Theta(\log n)$
- (D) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

Answer: (A, C, D)

Solution / Explanation:

By logarithmic identities:

$$f(n) = \log_2(n^8) = 8 \log_2 n = \Theta(\log n).$$

Meanwhile:

$$g(n) = (\log_2 n)^2 = \log_2 n \times \log_2 n.$$

Evaluating the ratio:

$$\frac{f(n)}{g(n)} = \frac{8 \log_2 n}{(\log_2 n)^2} = \frac{8}{\log_2 n} \xrightarrow{n \rightarrow \infty} 0.$$

Hence:

- $f(n)$ grows strictly slower than $g(n)$, meaning $f(n) = o(g(n)) \implies f(n) = \mathcal{O}(g(n))$ (A is True).
- $g(n) \neq \mathcal{O}(f(n))$ (B is False).
- $f(n) = 8 \log_2 n = \Theta(\log n)$ (C is True).
- The limit is 0 (D is True).

Practice Q9. Logarithmic Recurrence Tree with Division Step [GATE DA Standard • MCQ • 2 Marks] Consider the recurrence:

$$T(n) = T(\lfloor \sqrt{n} \rfloor) + 1, \quad T(2) = 1$$

What is the closed-form asymptotic behavior of $T(n)$?

- (A) $\Theta(\log n)$
- (B) $\Theta(\log \log n)$
- (C) $\Theta(\sqrt{n})$
- (D) $\Theta(\log^2 n)$

Answer: (B)

Solution / Explanation:

Let $n = 2^m$, so $m = \log_2 n$. Then $\sqrt{n} = (2^m)^{1/2} = 2^{m/2}$. Rewriting the recurrence in terms of $S(m) = T(2^m)$:

$$S(m) = S(m/2) + 1$$

Using Master Theorem for $S(m)$:

$$S(m) = \Theta(\log m)$$

Substituting back $m = \log_2 n$:

$$T(n) = \Theta(\log(\log_2 n)) = \Theta(\log \log n).$$

Practice Q10. Ternary Search vs Binary Search Comparisons [GATE DA Standard • MCQ • 1 Mark] Ternary search splits a sorted array into three equal parts using two midpoints and requires 2 comparisons per iteration in the worst case to isolate the subsegment of size $n/3$. Which of the following correctly compares the worst-case number of comparisons between Ternary Search (C_{ternary}) and Binary Search (C_{binary})?

- (A) Ternary search makes strictly fewer comparisons than binary search in the worst case.

- (B) Ternary search makes more comparisons than binary search in the worst case because $2 \log_3 n > \log_2 n$.
- (C) Both algorithms make the exact same number of comparisons for all n .
- (D) Ternary search has lower asymptotic time complexity $\mathcal{O}(\log_3 n) < \mathcal{O}(\log_2 n)$.

Answer: (B)

Solution / Explanation:

For binary search, worst-case comparisons $\approx \log_2 n \approx 1.00 \log_2 n$.

For ternary search:

$$2 \log_3 n = 2 \frac{\log_2 n}{\log_2 3} = \frac{2}{1.585} \log_2 n \approx 1.26 \log_2 n.$$

Since $1.26 > 1.00$, ternary search performs approximately 26% **more** comparisons in the worst case, despite reducing the subarray size faster.

End of Question Bank — GATE Data Science & Artificial Intelligence (DA)